Program: Master in Distributed Systems and Artificial Intelligence (SDIA)

Academic Year 2026-2027

LAB REPORT

Chaotic Text Generator

A test bench for part-of-speech HMM and trigram text generators

Field	Details
Student	Badr Eddine TOUBANI
Program	Master SDIA, second year
Institution	ENSET
Lab	Lab 03: Chaotic Text Generator
Data	Synthetic tagged corpus (1,200 sentences) and UD English EWT
Repository	https://github.com/toubani12/chaotic-text-generator
Date	November 2026

Table 0.1: Project information

TABLE OF CONTENTS

General introduction 4

Step 1: Data and models 5

1.1 The synthetic corpus 5

1.2 The hidden Markov model 5

1.3 Role-split states and the second-order extension 6

1.4 The trigram baseline 6

Step 2: The application 7

2.1 Architecture 7

2.2 Control panel and state-path graph 8

2.3 Generated sentences with tag chips 9

2.4 Data view 10

2.5 Tagger 12

2.6 Requirement coverage 12

Step 3: Results on the synthetic corpus 14

3.1 Prediction 14

3.2 Metrics at T = 1.0 14

3.3 Temperature 15

3.4 Failure classification 16

3.5 What state splitting changed, and what it did not 17

Step 4: Real text 19

4.1 Results 19

4.2 Why the gap 19

4.3 Suffix guessing (extension) 19

Synthesis and comparison 21

Conclusion and outlook 22

6.1 Project summary 22

6.2 Limits and future work 22

6.3 Operational recommendations 22

LIST OF ABBREVIATIONS AND TERMS

Term	Meaning
HMM	Hidden Markov Model: hidden states (here tags) that change by transition probabilities and emit words
POS	Part of speech: DET, ADJ, NOUN, VERB, PREP, PUNCT in the synthetic corpus
Role-split states	NOUN, ADJ and DET renamed by role: S (subject side), O (object side), P (prepositional phrase)
Viterbi	Algorithm that returns the most probable state sequence for a sentence
Forward-backward	Algorithm that gives the probability of each tag at each position (the confidence bar)
T	Temperature: sampling weights are count^(1/T)
k	Add-k smoothing constant, k = 0.01
OOV	Out of vocabulary: a word absent from the training vocabulary
Distinct-2	Different word bigrams divided by all word bigrams of a batch (diversity)
UD, EWT	Universal Dependencies, English Web Treebank (real text, 17 UPOS tags)
CSV	Comma-separated values file

Table 0.2: Abbreviations and terms used in this report

General introduction

A text generator produces sentences that have no reference answer. There is no gold sentence to hold "the lazy birds clean the apples" against, so reading 3 outputs and saying they look fine proves nothing. Quality has to be counted, on hundreds of sentences, with metrics that can be computed.

This lab builds a test bench for that job. A part-of-speech Hidden Markov Model (HMM) is estimated on a synthetic corpus of 1,200 tagged sentences, samples new sentences, and is compared with a trigram generator. The same HMM also runs backwards: given a sentence, the Viterbi algorithm recovers its tags.

The models came with the lab. Our work was the application around them, 5 extensions from the guide and the measurements. The report follows 4 steps:

1. **Data and models:** the corpus, the generators and the tagger.
2. **The application:** a Streamlit test bench, with its code on GitHub.
3. **Results on the synthetic corpus:** the 4 metrics, temperature, failure types and the effect of role-split states.
4. **Real text:** the same HMM on the Universal Dependencies English EWT treebank.

Step 1: Data and models

1.1 The synthetic corpus

The corpus has 1,200 sentences written by a small hand-made grammar with 6 tags. We split it **by document** before counting anything: 960 sentences for training and 240 for testing. A split by sentence would leak near-copies of training sentences into the test set.

Property	Value
Training sentences	960
Test sentences	240
Vocabulary	61 words
Tags	6 (DET, ADJ, NOUN, VERB, PREP, PUNCT)
Distinct tag sequences in training	16
Most frequent sequence	DET NOUN VERB DET NOUN PREP DET NOUN PUNCT (237 sentences)
"watch" as VERB / as NOUN	126 / 19

Table 1.1: The synthetic corpus after the split by document

Only 16 tag sequences exist in the training set. That small number is what makes the "valid tag pattern" metric possible: a generated sentence is valid when its tag sequence is one of them.

The word *watch* is the corpus' one ambiguity. It is a VERB in "the old children watch the birds at the kitchen ." and a NOUN in "the young children bring the watch behind the school .". Only the previous tag tells the two apart.

1.2 The hidden Markov model

The hidden states are the tags and the observations are the words. Training is counting: three tables (start, transition and emission probabilities) are filled from the training sentences, with no training loop.

```
P(words, tags) = P(t1) × Π P(ti | ti-1) × Π P(wi | ti)
```

To generate, the model samples a tag sequence first, then one word per tag. Temperature reshapes the counts before each draw:

```
weight(x) = count(x) ^ (1 / T)
```

Below T = 1 the frequent choices win more often. Above 1 the distribution flattens and rare choices come up. Tagging uses Viterbi in log space with add-k smoothing (k = 0.01), so an unseen word never gets probability 0 and long sentences don't underflow.

```
score = log P(t1) + Σ log P(ti | ti-1) + Σ log P(wi | ti)
```

1.3 Role-split states and the second-order extension

A first-order HMM only sees the previous tag. After a NOUN it can't know whether that noun was the subject (a verb must follow) or the object (a preposition or the full stop follows). In the training data the tag after NOUN is VERB 39.1% of the time, PUNCT 39.1% and PREP 21.9%, whatever the role.

Role-split states rename NOUN, ADJ and DET by position: S before the verb, O after it, P inside the prepositional phrase. The grammar stays the same. The model gets the one piece of memory it lacked, and the transitions out of the noun states become almost deterministic.

State	VERB	PREP	PUNCT
NOUN (plain)	0.39	0.22	0.39
NOUN_S	1.00	0	0
NOUN_O	0	0.66	0.34
NOUN_P	0	0	1.00

Table 1.2: Probability of the next tag after a noun state

As an extension from the guide we also built a **second-order HMM**, where the next tag depends on the two previous tags. It works as a generator and as a tagger. Section 3.2 shows why it doesn't fix the same problem.

1.4 The trigram baseline

The baseline is a word trigram that backs off to bigram and unigram counts when a context was never seen. It has no hidden states, so its sentences are tagged with Viterbi before they are scored. It's also the only model that can continue words typed by the user.

Step 2: The application

2.1 Architecture

The application is written in Streamlit. The model code stays in `src/pipeline.py` and contains no interface code; the interface lives in `app.py` and the `ui/` folder. One Lab object trains every model once at start-up and serves batches, metrics and tagging to the page.

File	Role
src/pipeline.py	HMM (plain, role-split, second order), trigram, Viterbi, forward-backward, suffix guesser, metrics, failure labels
src/evaluate.py	Evaluation on the synthetic corpus, 500 sentences per setting
src/evaluate_real.py	The same HMM on UD English EWT, with the most-frequent-tag baseline
app.py	Streamlit application: control panel, graph and the 7 tabs
ui/graph.py	SVG state-path graph
ui/components.py, ui/styles.py	Tables, tag chips, bars and the dark visual style
tests/test_pipeline.py	21 automated checks, including the guide's numbers at T = 1.0
report/build_report.py	One-page summary built from the live numbers

Table 2.1: Project files

We changed the provided pipeline without changing its results. Samplers keep their keys sorted and cache the cumulative weights per temperature, so a batch of 500 sentences takes a few milliseconds. A test checks that a seed still draws exactly the same sentences as the original code, and the guide's metrics at T = 1.0 come out identical.

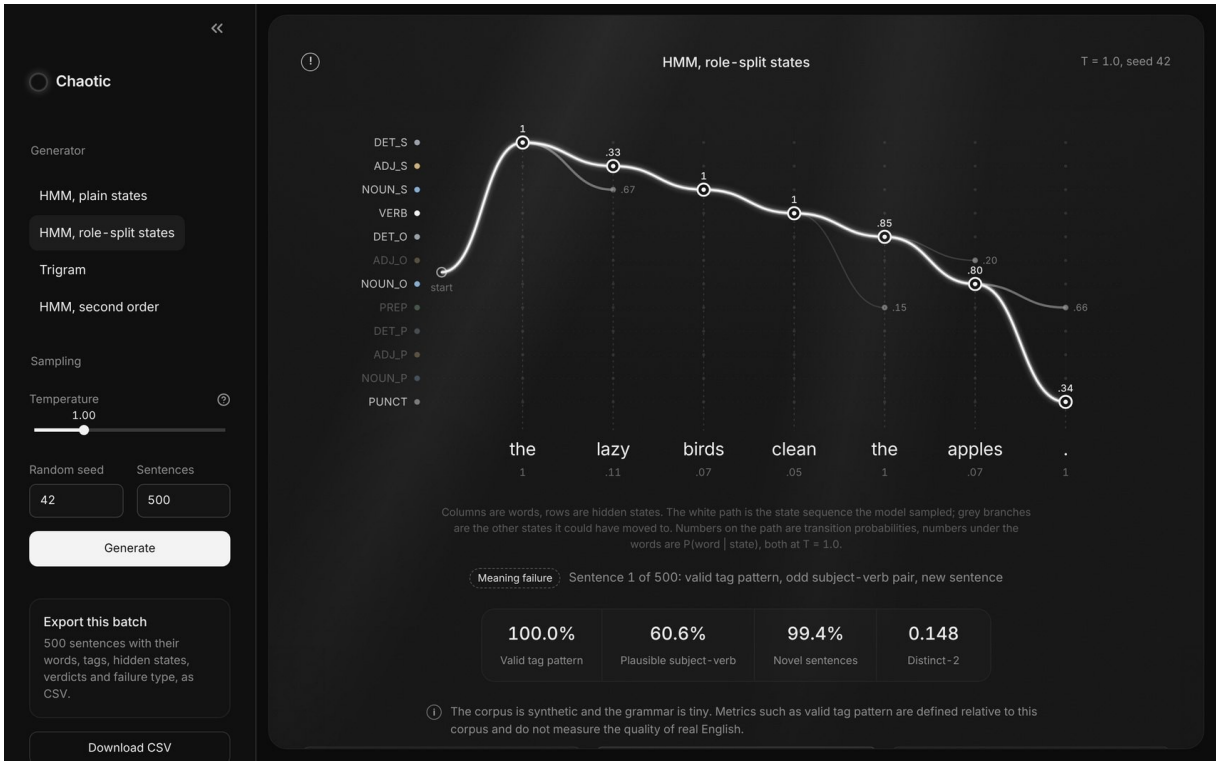


Figure 2.1: The application: control panel on the left, state-path graph and metrics on the right

2.2 Control panel and state-path graph

The sidebar holds the control panel: a model selector (plain HMM, role-split HMM, trigram and the second-order extension), the temperature from 0.3 to 3.0, the random seed and the number of sentences, from 10 to 500. **Generate** builds the batch and **Download CSV** exports it with words, tags, hidden states, verdicts and failure type.

The main panel draws one generated sentence as a graph. Columns are words and rows are hidden states. The white line is the path the model sampled. The grey branches are the states it could have moved to, with their probability at the current temperature, and the number under each word is P(word | state).



Figure 2.2: State-path graph of a role-split HMM sentence, with the branches not taken

Under the graph come the failure type of that sentence, the 4 metrics of the whole batch and the notice required by the guide. Buttons step through the batch or jump to the next structure, meaning or memorisation failure.

2.3 Generated sentences with tag chips

The Sentences tab lists the batch, one row per sentence. Each word carries its hidden state as a coloured chip, so the tag sequence reads directly under the words. A mark shows whether the tag pattern is valid, and the last column gives the failure type. Copied sentences get a white edge: that is the memorisation detector from the extension list.

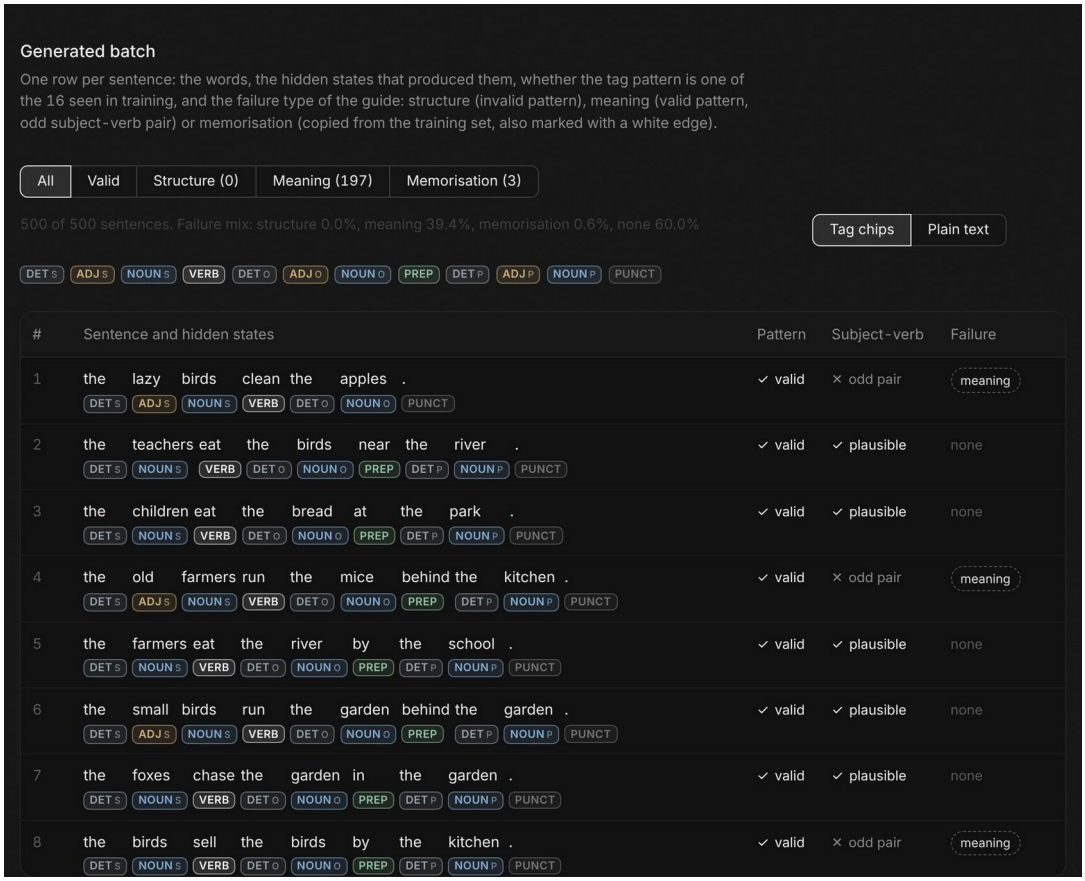


Figure 2.3: Sentences tab with tag chips, validity mark and failure type

2.4 Data view

The Data view tab profiles the batch against the training set: the failure mix, the share of each tag (with the training share as a grey marker), sentence lengths, a heat-map of tags by word position, the most frequent tag patterns, the words generated for each tag and the most common subject-verb pairs.

The position heat-map is the fastest check of structure. A grammatical batch reproduces the staircase of the training data: DET at position 1, ADJ or NOUN at 2, VERB at 3 or 4.

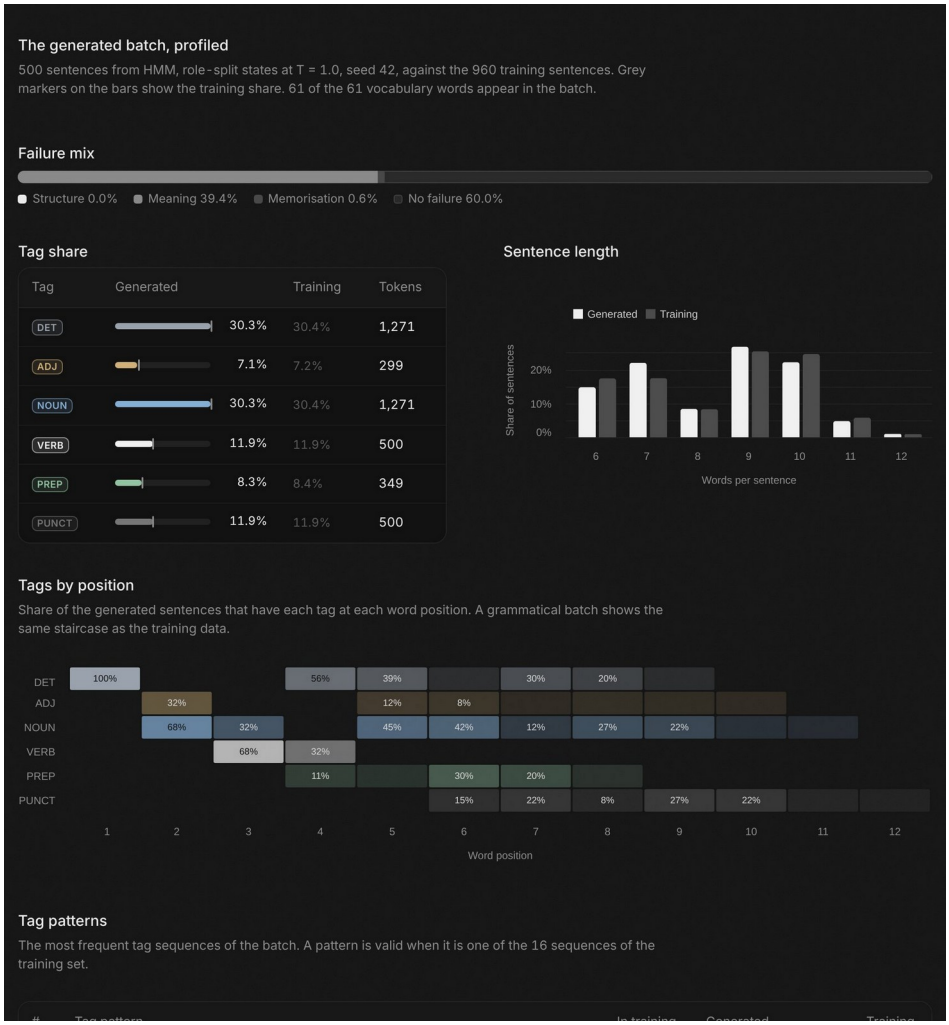


Figure 2.4: Data view: failure mix, tag share, sentence length and tags by position

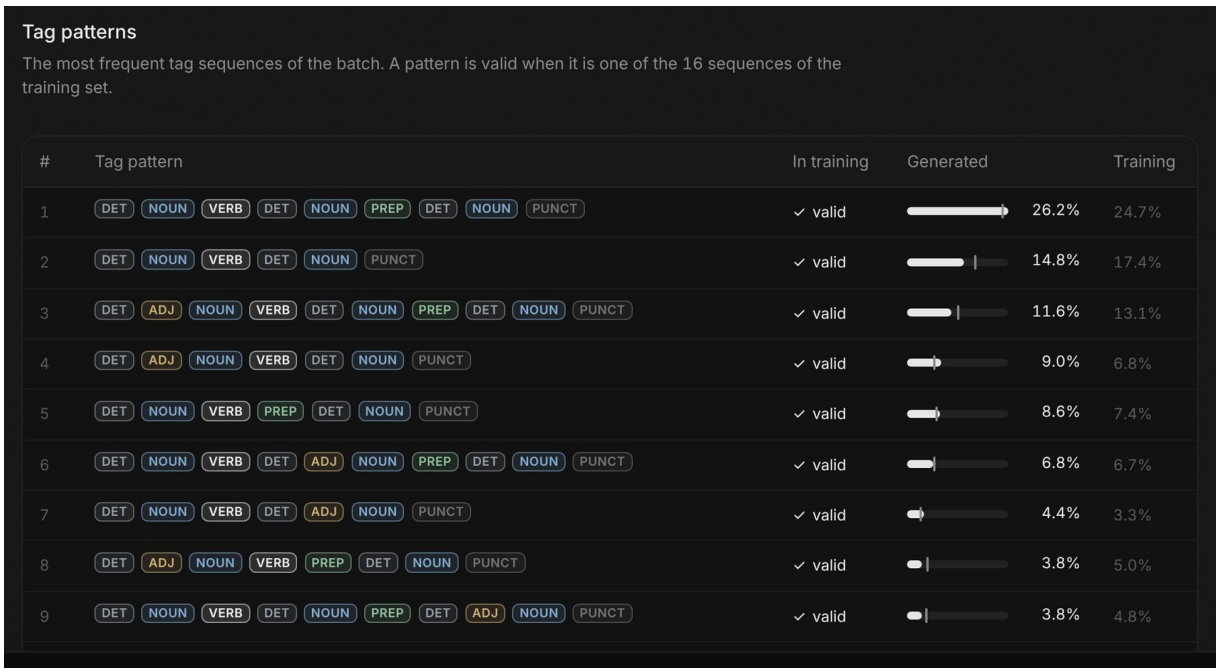


Figure 2.5: Data view: the most frequent tag patterns of the batch, compared with training

2.5 Tagger

The Tagger tab lower-cases any sentence, splits off punctuation and shows the Viterbi tag under each word, with a confidence bar computed by forward-backward. A word outside the 61-word vocabulary gets a dashed outline and a warning above the result. We tested empty input, symbols only, a 400-character paste and raw HTML: none of them crash the page.

The Viterbi path is drawn with the same graph as the main panel. For *watch* the branches make the ambiguity visible: after DET the path goes to NOUN, after a NOUN it goes to VERB.

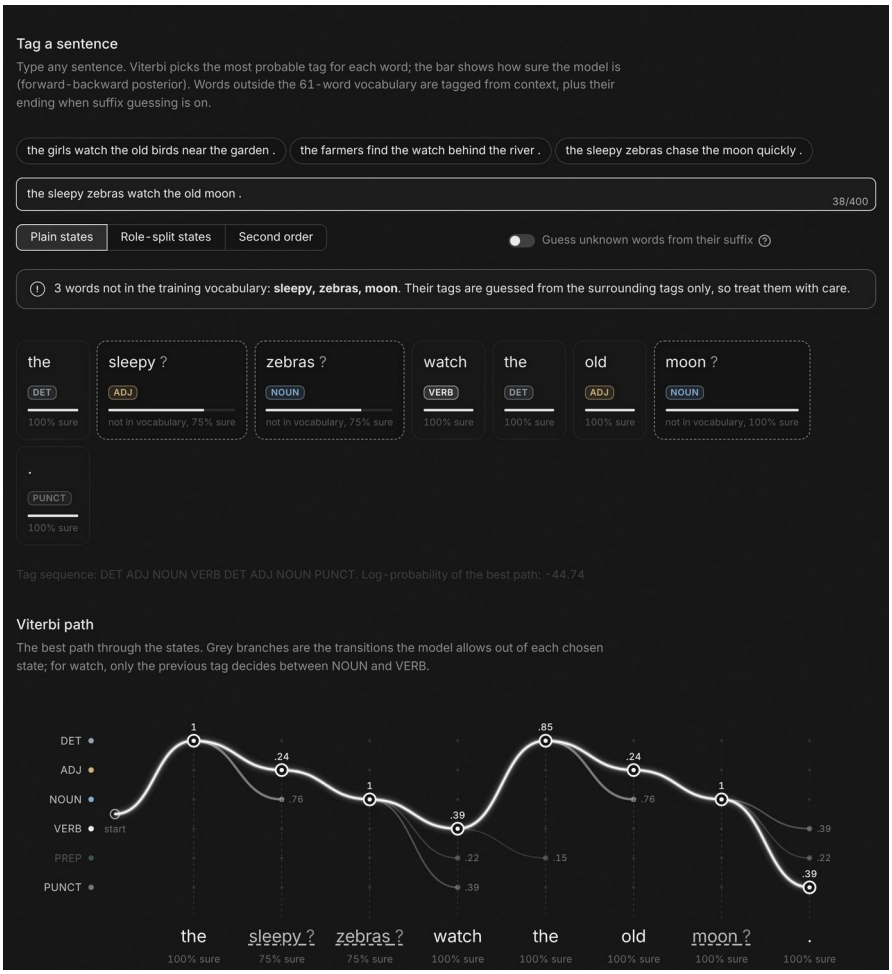


Figure 2.6: Tagger with 3 unknown words, the warning and the Viterbi path

2.6 Requirement coverage

Guide item	Where it is
A1 Control panel (model, temperature, seed, 10 to 500 sentences)	Sidebar
A2 Generate and a table with the hidden tags and a validity mark	Sidebar, Sentences tab
A3 Metrics panel (4 metrics)	Main panel, under the graph
A4 Tagger with an unknown-word warning, no crash	Tagger tab

Guide item	Where it is
A5 Download CSV	Sidebar
A6 Notice on the synthetic corpus	Main panel
Optional: comparison table and valid pattern against temperature	Compare models tab
B Real-data test	Real text tab, src/evaluate_real.py
Extensions: second-order HMM, suffix guessing, memorisation detector, heat-map	Sidebar, Tagger, Sentences, Transitions

Table 2.2: Where each requirement of the guide is met

Step 3: Results on the synthetic corpus

3.1 Prediction

Before running anything we expected the role-split HMM to produce the most valid tag patterns. Its states carry the role of each noun. The plain HMM forgets that role after every NOUN. The trigram sees two words, which covers a phrase but often breaks at the join between two phrases. Our predicted order was role-split HMM, then trigram, then plain HMM.

3.2 Metrics at T = 1.0

Model	Valid tag pattern	Plausible subject-verb	Novel sentences	Distinct-2
HMM, plain states	16.2%	14.8%	**99.8%**	0.198
HMM, role-split states	**100.0%**	60.6%	99.4%	0.148
Trigram	57.0%	**76.0%**	91.2%	0.105
HMM, second order (extension)	20.0%	13.8%	**99.8%**	**0.201**

Table 3.1: The 4 metrics at T = 1.0, 500 sentences, seed 42 (best value in bold)

The prediction held. The role-split HMM produces a valid pattern every time, the trigram 57.0% of the time and the plain HMM 16.2%.

Why the plain HMM scores low. After a NOUN it picks VERB, PREP or PUNCT with the same probabilities whether the noun is a subject or an object. A subject can then end the sentence before any verb ("the quiet school ."), and an object can take a second verb. A sentence has to win several of these draws in a row, and only 16.2% of them do.

Why the role-split HMM scores 100%. NOUN*S is always followed by VERB, NOUN*O by PREP or PUNCT and NOUN_P by PUNCT. Every path the model can sample is one of the 16 training patterns.

The second-order HMM reaches only 20.0%. The pair (DET, NOUN) looks the same for a subject and for an object, so two previous tags carry no information about the role. Tagging accuracy on the 240 test sentences is 100% for the three HMMs, and *watch* is tagged right 37 times out of 37.



Figure 3.1: A structure failure of the plain HMM: after NOUN it ends the sentence (probability 0.39) before any verb

3.3 Temperature

Model	T = 0.5	T = 1.0	T = 1.5	T = 2.5
HMM, plain states	23.8% / 0.149	16.2% / 0.198	20.4% / 0.213	15.4% / 0.217
HMM, role-split states	100.0% / 0.107	100.0% / 0.148	100.0% / 0.161	100.0% / 0.167
Trigram	63.6% / 0.095	57.0% / 0.105	48.2% / 0.115	44.2% / 0.112
HMM, second order	19.8% / 0.161	20.0% / 0.201	17.4% / 0.208	12.6% / 0.223

Table 3.2: Valid tag pattern / distinct-2 against temperature

Temperature trades safety for variety. For the trigram, valid patterns fall from 63.6% at T = 0.5 to 44.2% at T = 2.5 while distinct-2 rises from 0.095 to 0.112. The role-split HMM keeps 100% at every temperature; only its word choices spread out.

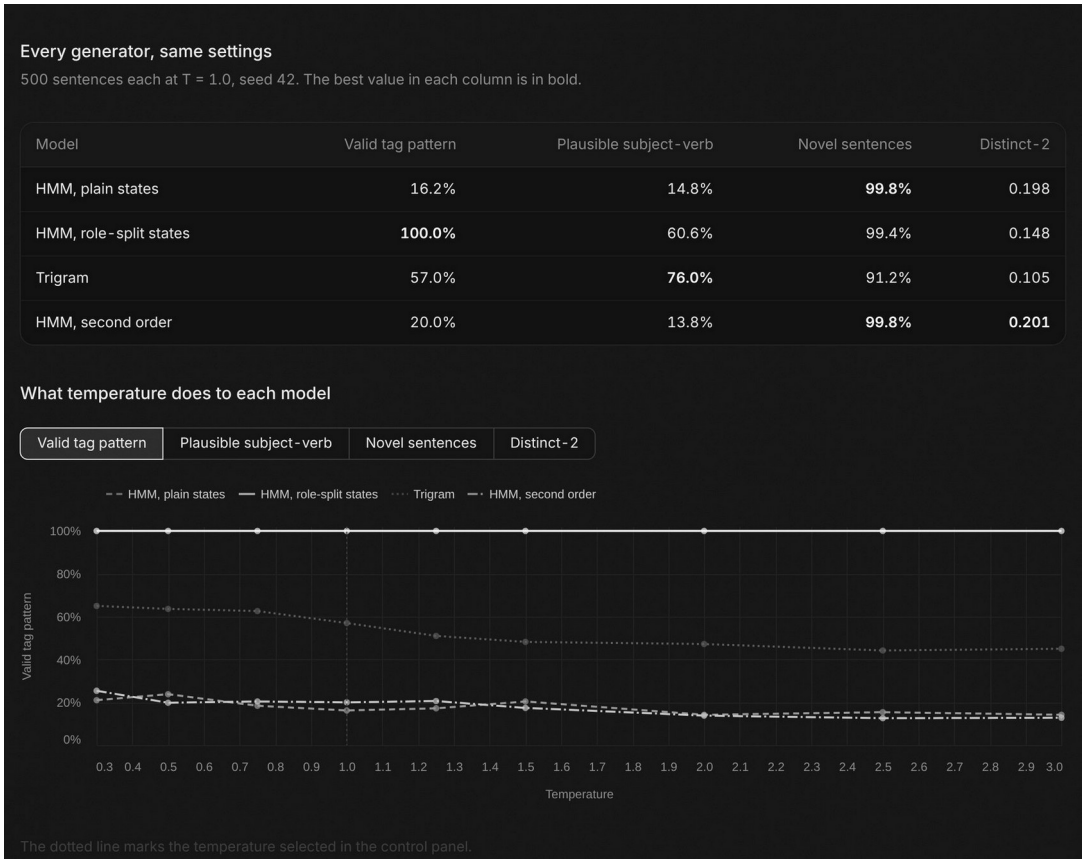


Figure 3.2: Compare models tab: the 4 generators at the same settings and valid pattern against temperature

Our choices for the two use cases of the guide:

- **Writing toy: role-split HMM at T = 1.5.** Structure stays at 100% and distinct-2 rises from 0.148 to 0.161. Odd word pairs are acceptable in a toy, even fun.
- **Autocomplete: trigram at T = 0.5.** It is the only model that continues the user's words, and a low temperature gives the most predictable continuation (63.6% valid patterns, 79.2% plausible pairs). Repeating a frequent phrase is fine for a suggestion.

3.4 Failure classification

Every generated sentence gets one of the guide's failure labels, or none. **Structure** means an invalid tag pattern. **Meaning** means a valid pattern whose first noun and verb never appear together in training. **Memorisation** means the sentence is copied from the training set.

Sentence	Model	Type	Cause
the quiet school .	Plain HMM, T = 1.0	Structure	After NOUN the model chose PUNCT without knowing that no verb had been produced
the happy market .	Plain HMM, T = 1.0	Structure	Same first-order memory problem

Sentence	Model	Type	Cause
the lazy birds clean the apples .	Role-split HMM, T = 1.0	Meaning	Noun and verb are drawn from their states independently; nothing links birds to clean
the old farmers run the mice behind the kitchen .	Role-split HMM, T = 1.0	Meaning	Same: states encode grammar, not meaning
the children visit the garden .	Trigram, T = 0.5	Memorisation	Low temperature follows the most frequent word path of a training sentence
the teachers eat the birds near the river .	Role-split HMM, T = 1.0	None	Valid pattern, seen subject-verb pair, new sentence

Table 3.3: Examples of each failure type and their cause

Model	Structure	Meaning	Memorisation	No failure
HMM, plain states	83.8%	11.6%	0.2%	4.4%
HMM, role-split states	0.0%	39.4%	0.6%	60.0%
Trigram	43.0%	0.2%	8.8%	48.0%
HMM, second order	80.0%	13.6%	0.2%	6.2%

Table 3.4: Failure mix over 500 sentences at T = 1.0

Each model fails in its own way. The plain HMM breaks structure in 83.8% of its sentences. The role-split HMM never does, but 39.4% of its sentences pair a noun with a verb never seen together. The trigram copies 8.8% of its sentences at T = 1.0, and those copies count against its novelty.

3.5 What state splitting changed, and what it did not

Splitting NOUN, ADJ and DET by role gives the first-order model the memory it lacked: where it is in the sentence. Valid tag patterns go from 16.2% to 100.0%, and plausible subject-verb pairs from 14.8% to 60.6% because subject nouns are now drawn only from subject positions.

It added no meaning. Nouns and verbs are still drawn independently, so the trigram stays ahead on plausibility (76.0%). Diversity drops (distinct-2 0.148 against 0.198), tagging accuracy was already 100%, and the states are written by hand for this one grammar: a relative clause would need new ones.

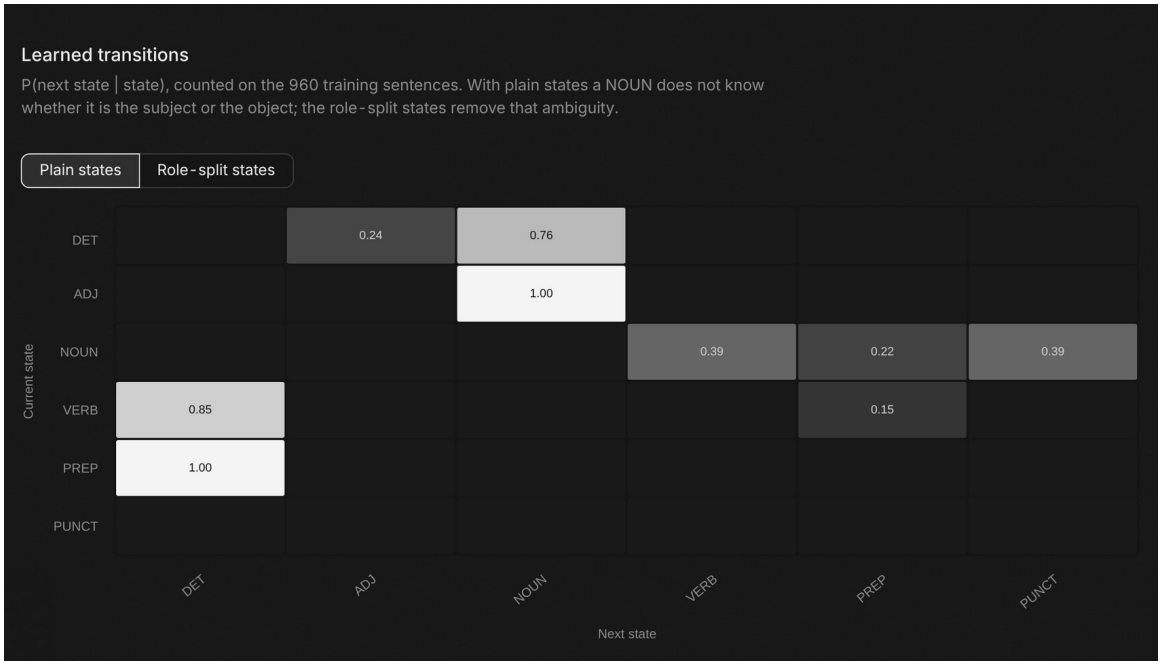


Figure 3.3: Learned transitions with plain states: the NOUN row splits between VERB, PREP and PUNCT

Step 4: Real text

4.1 Results

We ran the same HMM and Viterbi code on Universal Dependencies English EWT: real web text, 17 tags, 12,544 training and 2,077 test sentences, lower-cased.

Model	Accuracy	Known words	Unknown words	Whole sentence
HMM + Viterbi, synthetic corpus	100%	100%	no unknown words	100%
Most-frequent tag, EWT	86.2%	90.8%	28.6%	31.9%
HMM + Viterbi, EWT	88.0%	92.9%	28.2%	40.4%
HMM + Viterbi + suffix guessing, EWT (extension)	91.2%	93.3%	64.3%	49.0%

Table 4.1: Tagging accuracy, synthetic corpus against real text

4.2 Why the gap

The synthetic grammar is unambiguous once the previous tag is known, and every test word is in the vocabulary. Real text has neither property.

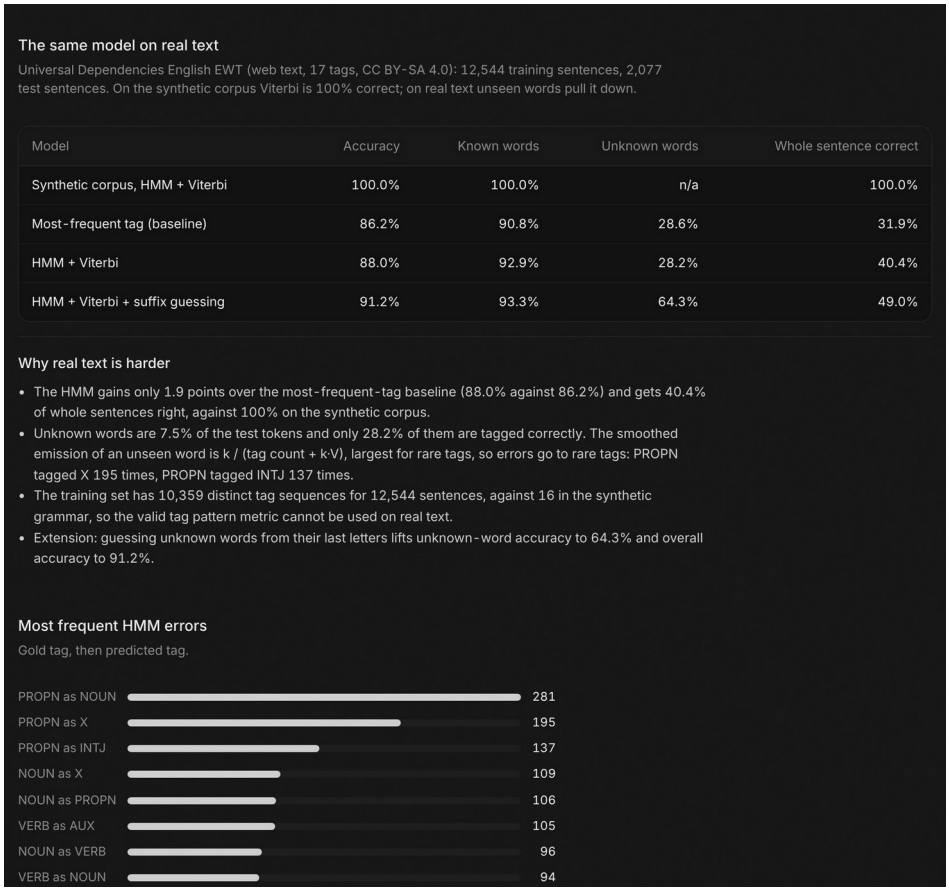
- The HMM gains only 1.9 points over the most-frequent-tag baseline (88.0% against 86.2%). It gets 40.4% of whole sentences right, against 100% on the synthetic corpus.
- Unknown words are 7.5% of the test tokens and only 28.2% of them get the right tag. The smoothed emission of an unseen word, k / (tag count + k·V), is largest for rare tags, so errors go to rare tags: PROPN tagged X 195 times and INTJ 137 times.
- The training set has 10,359 distinct tag sequences for 12,544 sentences, against 16 in the synthetic grammar. The valid tag pattern metric has no meaning on real text.

The guide reports 88.1% for the HMM. The current download of the treebank gives 88.0% with the unchanged code, so the 0.1-point gap comes from an update of the data.

4.3 Suffix guessing (extension)

For an unknown word we estimate P(tag | last 1 to 3 letters) on rare training words, which behave like unseen ones, and divide by P(tag). The learned rules make sense: words ending in -ly are adverbs 86% of the time, -ed is a verb 82% of the time and -ing a verb 68%.

Unknown-word accuracy goes from 28.2% to **64.3%**, overall accuracy from 88.0% to 91.2%, and whole sentences from 40.4% to 49.0%.



The same model on real text

Universal Dependencies English EWT (web text, 17 tags, CC BY-SA 4.0): 12,544 training sentences, 2,077 test sentences. On the synthetic corpus Viterbi is 100% correct; on real text unseen words pull it down.

Model	Accuracy	Known words	Unknown words	Whole sentence correct
Synthetic corpus, HMM + Viterbi	100.0%	100.0%	n/a	100.0%
Most-frequent tag (baseline)	86.2%	90.8%	28.6%	31.9%
HMM + Viterbi	88.0%	92.9%	28.2%	40.4%
HMM + Viterbi + suffix guessing	91.2%	93.3%	64.3%	49.0%

Why real text is harder

- The HMM gains only 1.9 points over the most-frequent-tag baseline (88.0% against 86.2%) and gets 40.4% of whole sentences right, against 100% on the synthetic corpus.
- Unknown words are 7.5% of the test tokens and only 28.2% of them are tagged correctly. The smoothed emission of an unseen word is k / (tag count + k·V), largest for rare tags, so errors go to rare tags: PROPN tagged X 195 times, PROPN tagged INTJ 137 times.
- The training set has 10,359 distinct tag sequences for 12,544 sentences, against 16 in the synthetic grammar, so the valid tag pattern metric cannot be used on real text.
- Extension: guessing unknown words from their last letters lifts unknown-word accuracy to 64.3% and overall accuracy to 91.2%.

Most frequent HMM errors

Gold tag, then predicted tag.

Figure 4.1: Real text tab: the results table, the analysis and the most frequent errors

Synthesis and comparison

Model	Valid	Plausible	Novel	Distinct-2	Main failure
HMM, plain states	16.2%	14.8%	**99.8%**	0.198	Structure
HMM, role-split states	**100.0%**	60.6%	99.4%	0.148	Meaning
Trigram	57.0%	**76.0%**	91.2%	0.105	Structure, memorisation
HMM, second order	20.0%	13.8%	**99.8%**	**0.201**	Structure

Table 5.1: The 4 generators side by side at T = 1.0 (best value in bold)

Each model wins a different column. The role-split HMM owns structure. The trigram owns plausibility, because it copies word pairs from training. The plain and second-order HMMs score highest on diversity, mostly because their sentences break in many different ways.

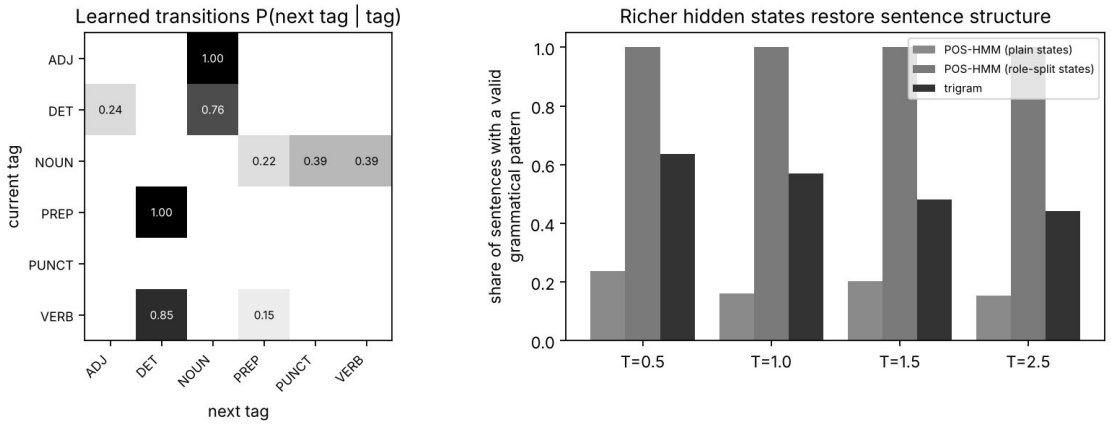


Figure 5.1: Output of src/evaluate.py: learned transitions and valid tag pattern against temperature

The two models worth using are the role-split HMM and the trigram. They fail in opposite directions:

Aspect	Role-split HMM	Trigram
Sentence structure	Always valid (100%)	57.0% valid at T = 1.0
Subject-verb plausibility	60.6%	76.0%
Copies training sentences	0.6%	8.8%
Effect of temperature	Structure stable, words vary	Structure degrades as T rises
Can continue a prompt	No	Yes
Best use	Writing toy (T = 1.5)	Autocomplete (T = 0.5)

Table 5.2: The two leading generators compared

Conclusion and outlook

6.1 Project summary

1. A Streamlit test bench that covers the 6 required features and the optional comparison table and temperature plot.
2. A state-path graph that shows, for every sentence, the hidden states, the branches not taken and their probabilities.
3. Tag chips in every table and a Data view tab that profiles each batch against the training set.
4. Failure labels (structure, meaning, memorisation) on every sentence, with a memorisation detector.
5. 4 extensions measured: second-order HMM (20.0% valid patterns), suffix guessing (64.3% on unknown EWT words), memorisation detector and transition heat-map.
6. 21 automated tests that pin the guide's numbers, so later changes can't silently break the results.

6.2 Limits and future work

- The data is synthetic and the grammar is tiny. Valid tag pattern and plausible subject-verb are defined relative to this grammar and say nothing about the quality of real English.
- The 100% tagging accuracy holds only because the synthetic grammar is unambiguous once the previous tag is known.
- The real-data test uses one lower-cased English web treebank. A first-order HMM is far from the best taggers, and the result doesn't carry over to other languages or domains.
- Next steps: condition the verb emission on the subject noun to attack meaning failures, add a relative-clause role state, and test capitalisation as a word-shape feature on EWT.
- A generator can reproduce its training data, so it should never be trained on private messages. Its text is fluent but has no intent, and it shouldn't be presented as written by a person.

6.3 Operational recommendations

Aspect	Recommendation
Evaluating a generator	Score at least 500 sentences per setting with a fixed seed; never judge from 3 examples
Choosing a model for structure	Role-split HMM: 100% valid patterns at every temperature
Choosing a model for autocomplete	Trigram at T = 0.5, and flag copied sentences
Decoding	Viterbi in log space with add-k smoothing; never multiply raw probabilities
Unknown words on real text	Add suffix guessing: +36.1 points on unknown words

Aspect	Recommendation
Deployment	Commit data/corpus.csv and results/ so the public app starts; keep data/ewt/ out of the repository

Table 6.1: Recommendations

Repository: https://github.com/toubani12/chaotic-text-generator. The application is deployed from this repository on Streamlit Community Cloud.